

1. List 3 real-world apps you use (like Zomato, Instagram, or Paytm) and describe one scenario in each where using serverless architecture would improve cost efficiency or scalability.

ANS

1) Zomato

- During peak lunch and dinner hours, Zomato can receive a large increase in restaurant searches and orders. Serverless functions can automatically scale based on incoming requests, reducing the need to keep servers running at full capacity all the time.

2) Instagram

- Instagram may experience sudden traffic increases when a popular post or reel becomes viral. Serverless architecture can process tasks such as image/video processing and notifications on demand, helping handle traffic spikes efficiently.

3) Paytm

- Paytm processes large numbers of transactions and API requests throughout the day. Serverless functions can handle event-driven tasks such as transaction notifications and payment-status processing, scaling automatically when request volume increases.

2. Create a simple REST API for a 'Favorite Songs' list using AWS Lambda and API Gateway — implement a Lambda function in your preferred language that returns a hardcoded list of 3 song names when accessed via an HTTP GET request.

ANS

- I created an AWS Lambda function named:
 - favoriteSongs
- The Lambda function returns a hardcoded list of three songs when it receives a GET request.

- **Lambda Function Code**

- import json code

❖ **Expected GET Response**

```
{  
  "songs": [  
    "Blinding Lights",  
    "Shape of You",  
    "Perfect"  
  ]  
}
```

3. Modify your Lambda function to accept a POST request that adds a new song name (sent in the request body) to an in-memory array, and returns the updated list.

Hint: Use a global variable in your Lambda function to simulate the array during the same execution (stateless between invocations).

[ANS](#)

- I modified the Lambda function to accept a POST request. The new song name is sent in the request body and added to an in-memory array.

❖ **Updated Lambda Code**

- import json code

❖ **POST Request Body**

```
{  
  "song": "Levitating"  
}
```

Expected Response

```
{  
  "message": "Song added successfully",  
  "songs": [  
    "Blinding Lights",  
    "Shape of You",  
    "Perfect",  
    "Levitating"  
  ]  
}
```

- The array is stored only in the Lambda execution environment. It is not permanent database storage. A later invocation may not contain the newly added song because Lambda is stateless.

4. Configure an API Gateway endpoint to trigger your Lambda function for both GET and POST requests, and test both endpoints using Postman or curl.

Hint: Make sure to set the correct HTTP method mappings in API Gateway.

ANS

- I created an **API Gateway HTTP API** and connected it to the favoriteSongs Lambda function.

❖ **API Routes**

HTTP Method Route Lambda

GET /songs favoriteSongs

POST /songs favoriteSongs

- The API Gateway endpoint was deployed and provided an HTTPS URL.

❖ **Test GET Using curl**

curl -X GET "https://YOUR_API_ID.execute-api.YOUR_REGION.amazonaws.com/songs"

❖ Expected response:

```
{
  "songs": [
    "Blinding Lights",
    "Shape of You",
    "Perfect"
  ]
}
```

❖ Expected response:

```
{
  "message": "Song added successfully",
  "songs": [
    "Blinding Lights",
    "Shape of You",
    "Perfect",
    "Levitating"
  ]
}
```

]

}

- I verified both GET and POST requests through API Gateway.

5. Pick a real-time data processing use case from an app you use (for example, live IPL score updates in Cricbuzz or real-time chat in WhatsApp). Explain in 3-4 lines how you would design a serverless workflow for this using Lambda, API Gateway, and any other AWS service if needed.

ANS

- A live cricket-score application such as Cricbuzz can use a serverless architecture to process score updates in real time.
- API Gateway can receive score updates, while AWS Lambda processes and validates the incoming data.
- The latest score can be stored in DynamoDB, and WebSocket API Gateway can push updates to connected users.
- This architecture can automatically scale when many users are watching a popular match simultaneously.

❖ **Serverless Workflow**

Cricket Data Source

↓

API Gateway

↓

AWS Lambda

↓

DynamoDB

↓

API Gateway WebSocket

↓

Connected Users